

■ Docker — Complete Notes

Spring Boot eCommerce Project | Interview Reference

1. What is Docker?

Docker is a platform that lets you **package your application + everything it needs** (JDK, dependencies, config, OS libraries) into a single unit called a **Container**. That container runs the same way on any machine — your laptop, teammate's machine, or production server.

Simple analogy:

Without Docker: "It works on my machine!" — classic problem Dev machine has Java 21, prod server has Java 17 → app breaks
With Docker: Container has Java 21 built-in → runs same everywhere "It works in my container!" → works everywhere ■

■ Interview Q: What is Docker in one line?

Docker is a containerization platform that packages an application and its dependencies into a lightweight, portable container that runs consistently across any environment.

2. Why Use Docker? (Purpose)

Problem Without Docker	Solution With Docker
"Works on my machine" — env differences	Same container runs everywhere
Setup takes hours per developer	docker-compose up — one command setup
Different Java/MySQL versions on servers	Container has exact versions built-in
Microservices hard to manage	Each service in its own container
Scaling is slow and manual	Spin up containers in seconds
Dependency conflicts between apps	Each container is fully isolated

■ For your project: One command starts Spring Boot + MySQL together. No manual setup needed.

3. Virtualization vs Docker (Containers)

Virtual Machine (Traditional):

Your Machine (Host OS - Windows) ■■■■ Hypervisor (VMware / VirtualBox) ■■■■ VM 1: Full OS (Ubuntu) + App 1 ← 2GB RAM just for OS! ■■■■ VM 2: Full OS (CentOS) + App 2 ← 2GB RAM just for OS! ■■■■ VM 3: Full OS (Windows) + App 3 ← 2GB RAM just for OS! Each VM = Full operating system copy Startup time: 1-2 minutes Size: GBs

Docker (Containers):

Your Machine (Host OS - Windows) ■■■■ Docker Engine ■■■■ Container 1: App 1 only ← shares Host OS kernel! ■■■■ Container 2: App 2 only ← shares Host OS kernel! ■■■■ Container 3: MySQL only ← shares Host OS kernel! Each Container = App + dependencies only (NO full OS copy) Startup time: seconds Size: MBs

Feature	Virtual Machine	Docker Container
OS	Full OS per VM (GBs)	Shares host OS kernel (MBs)
Startup	1-2 minutes	Seconds
Size	GBs	MBs
Isolation	Complete (hardware level)	Process level
Performance	Slower (OS overhead)	Near native speed
Use case	Full OS isolation needed	App packaging + microservices

■ Interview Q: How is Docker different from a Virtual Machine?

VMs include a full OS per instance — heavy, slow to start, GBs in size. Docker containers share the host OS kernel — lightweight, start in seconds, MBs in size. VMs provide hardware-level isolation. Containers provide process-level isolation. For microservices, Docker is preferred due to speed and resource efficiency.

4. Docker Architecture

Docker Client (CLI) Docker Hub (Registry) ■ ■ ■ docker commands ■ images stored here ↓ ■ Docker Daemon (dockerd) ←■■■■■■■■■■■■■■■■■■■■ ■ ■■■ Images (blueprints) ■■■ Containers (running instances) ■■■ Volumes (persistent storage) ■■■ Networks (container communication)

Component	What it is	Analogy
Docker Client	CLI you type commands into	TV Remote
Docker Daemon	Background service that does the work	TV itself
Docker Image	Blueprint/template to create containers	Recipe
Docker Container	Running instance of an image	Cooked dish
Docker Hub	Cloud registry to store/share images	App Store
Docker Volume	Persistent storage for containers	External hard drive
Docker Network	Communication between containers	Internal WiFi

■ Interview Q: Explain Docker architecture.

Docker uses a client-server architecture. Docker Client sends commands to Docker Daemon via REST API. Daemon manages images, containers, volumes and networks. Images are pulled from Docker Hub (registry). Containers are running instances of images. Volumes persist data outside container lifecycle.

5. Dockerfile — What is it?

A Dockerfile is a text file with step-by-step instructions to build a Docker Image. Think of it as a recipe — Docker reads it top to bottom and creates your image.

Your project's Dockerfile explained:

[illegible]

Instruction	Purpose
FROM	Base image to start from
WORKDIR	Set working directory inside container
COPY	Copy files from local machine to container
RUN	Execute command during BUILD time
EXPOSE	Document which port container listens on
ENTRYPOINT	Command to run when container STARTS
ENV	Set environment variables
ARG	Build-time variables (not available at runtime)
VOLUME	Create mount point for persistent storage

■ Interview Q: What is multi-stage Dockerfile and why use it?

Multi-stage Dockerfile uses multiple FROM instructions. Stage 1 builds the app using Maven/JDK (heavy). Stage 2 only runs it using JRE (lightweight). Final image contains only Stage 2 — no Maven, no source code, no build tools. Result: smaller (700MB → 200MB), more secure image since attack surface is reduced.

■ Interview Q: Why copy pom.xml separately before src?

Docker builds in layers and caches each layer. `pom.xml` changes rarely, `src` changes often. By copying `pom.xml` first and running `mvn dependency:resolve`, Docker caches the dependencies layer. Next build — if only `src` changed, Docker skips re-downloading dependencies. Saves 2-3 minutes per build.

6. Image vs Container vs Hub

Docker Image Docker Container  Blueprint /
Template Running instance of Image Read-only Read-write (adds layer on top) Stored on disk Lives
in memory (while running) Like a CLASS in Java Like an OBJECT in Java Built from Dockerfile
Created from Image via docker run Can create many containers One per docker run command Docker Hub
 Cloud registry (like GitHub but for Docker images) Public images: mysql,
nginx, openjdk, maven Private images: your company's images Push your image: docker push
yourname/ecom-app Pull image: docker pull mysql:8.0

```
# Image commands
docker images # list all images
docker pull mysql:8.0 # download from DockerHub
docker build -t ecom-app . # build from Dockerfile
docker rmi ecom-app # remove image
docker tag ecom-app yourname/ecom-app # rename/tag image
docker push yourname/ecom-app # push to DockerHub #
Container commands
docker run -p 8080:8080 ecom-app # create + start container
docker ps # list running containers
docker ps -a # list all containers (including stopped)
docker stop # stop container
docker rm # remove container
docker logs # see container logs
docker exec -it bash # enter container terminal
```

■ Interview Q: Difference between Image and Container?

Image is a static read-only blueprint — like a class in Java. Container is a running instance of that image — like an object. One image can create many containers. Image is built once, containers are created and destroyed frequently.

7. Docker Volumes — Persistent Storage

Containers are stateless — when a container stops, all data inside it is LOST. Volumes solve this by storing data outside the container lifecycle.

```
Without Volume: Container starts → MySQL stores data inside container docker-compose down → container deleted → ALL DATA GONE! ■ With Volume: Container starts → MySQL stores data in named volume (outside container) docker-compose down → container deleted → DATA SAFE in volume ■ docker-compose up → new container mounts same volume → data still there ■
```

In your docker-compose.yml:

```
mysql: volumes: - mysql_data:/var/lib/mysql # mysql_data = named volume (Docker manages location) # /var/lib/mysql = where MySQL stores data INSIDE container # Docker maps these together volumes: mysql_data: # declare the named volume here
```

Volume commands:

```
docker volume ls # list all volumes docker volume inspect mysql_data # see volume details + location docker volume rm mysql_data # delete volume (wipes data!) docker-compose down -v # stop + delete volumes
```

Type	Example	Use When
Named Volume	mysql_data:/var/lib/mysql	DB data, Docker manages location ■
Bind Mount	./logs:/app/logs	Dev: share local folder with container
tmpfs Mount	tmpfs: /tmp	Temp data, never persisted

■ Interview Q: Why are volumes needed in Docker?

Containers are ephemeral — data inside them is lost when container stops. Volumes persist data outside the container lifecycle on the host machine. For databases like MySQL, volumes are essential — without them every restart wipes all data.

8. Docker Compose — Multi-Container Management

Docker Compose lets you define and run multiple containers together using a single YAML file. One command starts your entire application stack.

Your docker-compose.yml fully explained:

```
services: # define all containers app: # Spring Boot container build: context: . # look for
Dockerfile in current folder dockerfile: Dockerfile # use this specific Dockerfile ports: -
"8080:8080" # hostPort:containerPort # localhost:8080 → container:8080 # format:
"YOUR_MACHINE_PORT:CONTAINER_PORT" environment: # override application.properties -
SPRING_PROFILES_ACTIVE=prod # activates application-prod.properties -
SPRING_DATASOURCE_URL=jdbc:mysql://mysql:3306/ecom_db # NOTE: 'mysql' = service name below, NOT
localhost! # Inside Docker network, containers talk via SERVICE NAME depends_on: - mysql # start
mysql FIRST, then app restart: on-failure # restart if app crashes (race condition fix) mysql: #
MySQL container image: mysql:8.0 # pull from DockerHub directly environment: MYSQL_ROOT_PASSWORD:
root # set root password MYSQL_DATABASE: ecom_db # auto-create this DB on first start ports: -
"3307:3306" # expose on 3307 (avoids local MySQL conflict) volumes: - mysql_data:/var/lib/mysql #
persist DB data volumes: mysql_data: # declare named volume
```

Ports explained in detail — how -p works:

```
ports: - "8080:8080" # hostPort:containerPort Your Laptop Docker Container ██████████
██████████ localhost:8080 ██████████→ container:8080 binding Your Laptop Docker
Container ██████████ ████████████ localhost:3307 ██████████→ mysql container:3306
binding # MySQL inside container always uses 3306 # But outside (your machine) you access it on
3307 # Avoids conflict with your local MySQL on 3306 # On a PRODUCTION SERVER: server_ip:8080
██████████→ container:8080 # Replace localhost with server IP
```

Docker Compose commands:

```
docker-compose up --build # build images + start all containers
docker-compose up --build -d # same but in background (detached)
docker-compose down # stop + remove containers
docker-compose down -v # stop + remove containers + volumes
docker-compose stop # stop containers (keep them)
docker-compose start # start stopped containers
docker-compose logs -f app # follow app logs live
docker-compose ps # list containers status
docker-compose restart app # restart specific service
```

■ Interview Q: Why use 'mysql' instead of 'localhost' in JDBC URL?

Inside Docker, each container has its own network namespace. 'localhost' inside the app container refers to the app container itself — not MySQL. Docker Compose creates a shared network where containers communicate via SERVICE NAME. So 'mysql' resolves to the MySQL container's IP automatically within the Docker network.

■ Interview Q: What does depends_on do?

depends_on controls startup ORDER — MySQL starts before the app. However it only waits for the container to START, not for MySQL to be READY. That's why we add restart: on-failure — if app crashes because MySQL isn't ready yet, Docker restarts it automatically.

9. How It Works Internally — docker-compose up --build

Step 1: Docker reads docker-compose.yml Finds 2 services: app, mysql Step 2: For 'mysql' service image: mysql:8.0 → checks local cache Not found → pulls from DockerHub Creates mysql container Sets environment variables (root password, DB name) Mounts volume mysql_data to /var/lib/mysql Exposes port 3307:3306 Step 3: For 'app' service (build: .) Reads Dockerfile line by line: FROM maven:3.9-amazoncorretto-21 AS builder → pulls maven image from DockerHub WORKDIR /app → creates /app directory inside container COPY pom.xml . → copies your pom.xml → checks cache layer → if pom.xml unchanged, uses cached layer (fast!) COPY src ./src → copies your source code RUN mvn clean package -DskipTests → runs Maven inside container → downloads dependencies → compiles Java code → creates /app/target/ecom-app.jar FROM amazoncorretto:21 → starts fresh lightweight image (Stage 2) → discards Stage 1 (Maven + source code gone) COPY --from=builder /app/target/*.jar app.jar → copies ONLY the JAR from Stage 1 EXPOSE 8080 → documents port (informational) ENTRYPOINT ["java", "-jar", "app.jar"] → sets startup command Step 4: Creates Docker network 'ecomapplication_default' Both containers join this network Can now reach each other by service name Step 5: Starts mysql container first (depends_on) MySQL initializes, creates ecom_db database Step 6: Starts app container Runs: java -jar app.jar Spring Boot starts, connects to mysql:3306 Hibernate creates tables (ddl-auto=update) App ready on port 8080 Step 7: You hit localhost:8080 Docker routes to container:8080 Response comes back

■ Interview Q: What happens when you run docker-compose up --build?

Docker reads compose file, pulls/builds images, creates a shared network, starts containers in dependency order (MySQL first), mounts volumes, sets environment variables, and runs entrypoint commands. Containers communicate via service names on the shared Docker network.

10. Ways to Dockerize Spring Boot App

Way 1 — Basic Dockerfile:

```
FROM amazoncorretto:21 WORKDIR /app COPY target/ecom-app.jar app.jar # copy pre-built JAR EXPOSE 8080 ENTRYPOINT ["java", "-jar", "app.jar"] # Commands: mvn clean package -DskipTests # build JAR first manually docker build -t ecom-app . # build image docker run -p 8080:8080 ecom-app # run container
```

■ Simple | ■ Must build JAR manually before docker build

Way 2 — Multi-Stage Dockerfile (Production Standard):

```
FROM maven:3.9-amazoncorretto-21 AS builder WORKDIR /app COPY pom.xml . COPY src ./src RUN mvn clean package -DskipTests # Maven runs INSIDE Docker FROM amazoncorretto:21 WORKDIR /app COPY --from=builder /app/target/*.jar app.jar EXPOSE 8080 ENTRYPOINT ["java", "-jar", "app.jar"] # Command: docker build -t ecom-app . # Docker handles everything
```

■ No manual JAR build | ■ Small final image | ■ Used in CI/CD

Way 3 — Spring Boot Maven Plugin (Buildpacks):

```
# No Dockerfile needed! # Just run: mvn spring-boot:build-image -DskipTests # How it works internally: mvn spring-boot:build-image | v Spring Boot Maven Plugin | v Calls Paketo Buildpacks | v DETECT PHASE: - Found pom.xml? → Maven Java project - Found spring-boot? → Spring Boot app - java.version=21? → Use JDK 21 | v BUILD PHASE: - Downloads correct JDK buildpack - Downloads Spring Boot buildpack - Compiles and packages app - Creates optimized layered image | v docker.io/library/ecom.application:0.0.1-SNAPSHOT ■ # Run: docker run -p 8080:8080 ecom.application:0.0.1-SNAPSHOT
```

■ No Dockerfile | ■ Auto-optimized layers | ■ Less control

Way 4 — docker-compose (Full Stack):

```
# Runs App + MySQL + any other services together docker-compose up --build ■ One command starts everything ■ Containers talk to each other via service names ■ Volumes persist DB data ■ Environment variables override properties
```

■ Best for development and production deployment

Way	Dockerfile	Builds JAR?	MySQL Included	Best For
Basic Dockerfile	Yes	No (manual)	No	Learning
Multi-Stage Dockerfile	Yes	Yes (inside)	No	CI/CD, Production
Maven Plugin/Buildpacks	No	Yes	No	Quick builds
docker-compose	Yes	Yes	Yes	Full stack dev/prod

11. Buildpacks — How They Work Internally

Buildpacks automatically detect what kind of app you have and build an optimized Docker image without any Dockerfile. Used by Heroku, Cloud Foundry, and Spring Boot Maven Plugin.

Detection Phase — how Buildpacks identify your app:

```
Each buildpack has a detect script: Java Buildpack detect script (internally): if [ -f "pom.xml" ] || [ -f "build.gradle" ]; then echo "Java project detected!" && exit 0 # █ I'll handle this else exit 1 # █ not my job, skip fi Node Buildpack detect script: if [ -f "package.json" ]; then exit 0 # █ else exit 1 # █ fi Python Buildpack: if [ -f "requirements.txt" ]; then exit 0; else exit 1; fi → Multiple buildpacks run detect → Whichever passes → that buildpack handles the build → For your project: Java + Maven + SpringBoot buildpacks all pass █
```

What Buildpacks detects from YOUR pom.xml:

pom.xml contains: 21 → use JDK 21 spring-boot-starter-parent → Spring Boot app
spring-boot-maven-plugin → executable JAR mysql-connector-j → MySQL app Buildpacks reads all this and: 1. Downloads exact JDK 21 layer 2. Downloads Spring Boot optimized run layer 3. Packages JAR in optimized layers: - Layer 1: JDK (rarely changes) - Layer 2: Spring Boot loader (rarely changes) - Layer 3: Dependencies (changes occasionally) - Layer 4: Your code (changes frequently) 4. Creates final image with these layers Result: Faster future builds (only changed layers rebuild)

Buildpacks vs Dockerfile comparison:

Feature	Dockerfile	Buildpacks
Setup	Write manually	Zero setup
Detection	You specify everything	Auto-detects language/framework
Optimization	Manual layering	Auto-optimized layers
Image size	Depends on your skill	Automatically minimal
Control	Full control	Limited control
Maintenance	You update Dockerfile	Buildpacks auto-update
Security patches	Manual	Automatic via buildpack updates

█ Interview Q: What are Buildpacks and how do they work?

Buildpacks are scripts that automatically detect your application type by checking project files (pom.xml for Java, package.json for Node etc.) and build an optimized Docker image without any Dockerfile. They run in two phases: detect (identify app type) and build (create optimized layered image). Spring Boot Maven Plugin uses Paketo Buildpacks internally when you run `mvn spring-boot:build-image`.

12. Complete Docker Command Reference

Image Commands:

```
docker images # list all local images docker pull mysql:8.0 # download image from DockerHub
docker build -t myapp . # build image from Dockerfile docker build -t myapp:v1.0 . # build with specific tag
docker rmi myapp # remove image docker tag myapp username/myapp:latest # tag for DockerHub
docker push username/myapp:latest # push to DockerHub docker image prune # remove unused images
```

Container Commands:

```
docker run -p 8080:8080 myapp # run container docker run -p 8080:8080 -d myapp # run in background
docker run -p 8080:8080 -e KEY=VALUE myapp # with env variable docker ps # list running containers
docker ps -a # list all containers docker stop # stop container docker start # start stopped container
docker rm # remove container docker logs # view logs docker logs -f # follow logs live
docker exec -it bash # enter container shell docker inspect # full container details
```

Docker Compose Commands:

```
docker-compose up --build # build + start all docker-compose up --build -d # background mode
docker-compose down # stop + remove containers docker-compose down -v # + remove volumes (wipes DB!)
docker-compose stop # stop (keep containers) docker-compose start # start stopped containers
docker-compose restart # restart all docker-compose logs -f app # follow app logs docker-compose ps # status of all services
docker-compose exec app bash # enter app container shell
```

Volume Commands:

```
docker volume ls # list volumes docker volume inspect mysql_data # volume details docker volume rm mysql_data # delete volume
docker volume prune # remove unused volumes
```